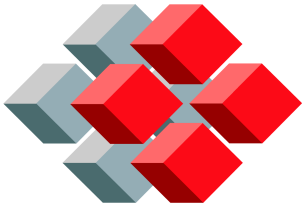
ETH
Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

CSCS
Swiss National Supercomputing Centre



Hybrid MPI/OpenMP Parallel Programming

Introduction to Parallel Programming
National Supercomputing Service, CSCS



Agenda

- What is hybrid programming?
- Why use it?
- Strategy and short examples
- Performance considerations
- Questions



What is hybrid programming?

- Hybrid programming is the use of different programming models to exploit different levels of parallelism
  - Increase scaling
  - Address load balance issues
  - Decrease communications overhead
  - Examples:
    - MPI + OpenMP on multicore systems
    - MPI + OpenCL on GPU enabled systems



Why might you use it?

- Reduce time to solution
  - Reduce number of MPI messages passed
    - Also reduce the amount of MPI communication within a node.
    - Best used when you are network-bandwidth limited
  - Number of MPI tasks is limited by problem
  - Unexploited parallelism not easily handled by your current MPI decomposition.



Hybrid programming: Best of both worlds?

- MPI
  - Explicit parallelism, which is often very efficient.
  - Inter-node communication.
  - Program state synchronization is explicit
  - Optimized collective routines available.
  - Usually no problems with data placement.
- OpenMP
  - High performance intra-node threading.
  - Easy to implement.
  - Makes better use of shared memory.
  - Program state synchronization is implicit.
  - Dynamic and run-time scheduling of work.



Common Example of a Hybrid Code

```
 PROGRAM
 CALL MPI_INIT (ierr)
 CALL MPI_COMM_RANK (…)
 CALL MPI_COMM_SIZE (…)
! …  some computation and MPI communication
!$OMP PARALLEL DO
       DO I = 1, N
              CALL Physics()
        END DO
!$OMP END PARALLEL DO
!…  some computation and MPI communication
 CALL MPI_FINALIZE (ierr)
 end
```



A Hybrid Parallelization Strategy

- Decompose at a high level using MPI
- Add OpenMP to exploit loop-level and/or task-level parallelism
- Use MPI outside of OpenMP parallel regions
  - Only master thread performs MPI comms.
- Could use MPI within a parallel region



Using MPI in threaded regions

- 'Normally/classicaly', MPI calls are not placed within parallel regions.
  - This is because MPI-1 was not *thread safe*.
- However, this creates load imbalance
  - 1 core does communication
  - N-1 cores are idle
- Possible to use *thread-safe* MPI to, perhaps, overlap communication and computation



Example (master calls MPI)

```
!$OMP PARALLEL
      !$OMP DO
      DO I = 1, N
            CALL Physics()
      END DO
      !OMP END DO
!master thread does MPI
      !$OMP BARRIER
      !$OMP MASTER
      CALL MPI_IRECV()
      CALL MPI_SEND()
      !$OMP END MASTER
      !$OMP BARRIER
! need barrier  since no implicit
! barrier for master directive

!$OMP END PARALLEL DO
```



Example (single calls MPI)

```
!$OMP PARALLEL
       !$OMP DO
       DO I = 1, N
              CALL Physics()
       END DO
       !OMP END DO
!any individual thread does MPI
       !$OMP BARRIER
       !$OMP SINGLE
       CALL MPI_IRECV()
       CALL MPI_SEND()
       !$OMP END SINGLE
! No need for barrier  since implicit
! barrier for single directive

!$OMP END PARALLEL DO
```



Example (DIY)

```
!$OMP PARALLEL
      thrdID = omp_get_thread_num()
      IF( thrdID /= commThread )THEN
      !$OMP DO
      DO I = 1, N
            CALL Physics()
       END DO
      !OMP END DO
      ELSE
            CALL MPI_IRECV()
            CALL MPI_SEND()
      END IF
      !$OMP BARRIER
!$OMP END PARALLEL DO
```

Example (nested parallelism)

```
!$OMP PARALLEL
      thrdID = omp_get_thread_num()
      IF( thrdID > n0)THEN
      !$OMP PARALLEL NUM_THREADS(n)
            ! Do some work
      !OMP END PARALLEL
      ELSE
            !do some other work
      END IF
!$OMP END PARALLEL
```



Initializing threaded MPI calls

- In order to use MPI calls in threaded regions, you must use a special version of the `MPI_Init()` routine
  - `MPI_Init_Thread( int *argc, char ** argv, int required, int * provided )`
- `required` = level of thread support desired.
- `provided` = level of thread support provided.



Types of MPI thread support

- `MPI_THREAD_SINGLE`
  - Only one thread will execute an MPI call.
- `MPI_THREAD_FUNNELLED`
  - Only the master thread will execute an MPI call.
- `MPI_THREAD_SERIALIZED`
  - Multiple threads may make MPI calls, but only one at a time.
- `MPI_THREAD_MULTIPLE`
  - Multiple threads may make MPI calls.



Things to consider

- If the loops/sections that you want to parallelize don't scale, hybrid programming isn't going to help you
  - See content on OpenMP optimization
- If your loops are memory-bandwidth bound then it's likely that this type of method won't help you.
  - Can be a problem with codes ported from vector machines that use a lot of arrays to store values instead of computing what you need from what you have.
  - Just like MPI, you want to limit the amount of communication that you have to do.

More things to consider

- Thread creation and synchronization has a lot of overhead:
  - Instead of parallelizing many loops in a subroutine, consider changing code and data structures so that you are parallelizing a a single loop

Questions?

- Do you have plans to hybridize your code?
- What are you trying to do?
- Did you try it and it didn't do much for you
  - Why?